

What is parallelized in the Lie-series commutation kernel

Status: partially superseded (2026-09-05). The algebra here — feasible pairs, content homogeneity, the anagram-unit partition — is unchanged and still the foundation; see `parallelism-in-the-fold.typ` for the canonical treatment. The execution mechanics changed since: bundles are now cut per unit under a single-phase direct-add sweep (no two-phase term/fan-in), balance by unit pair-weight, and the gating/ticket machinery is cached per exact operand supports. Numbers below are as of 2026-09-01.

Your model is right for the partition. The pairs to commute are grouped into *anagram units*, and units have no dependencies — sweeping them is embarrassingly parallel. What the “2t = ½ runtime” intuition misses is (a) the serial prologue and dispatch *around* the sweep, and (b) the sweep’s own parallel *efficiency* — a memory-system property, not a dependency property.

The work being divided: feasible pairs, grouped by anagram class

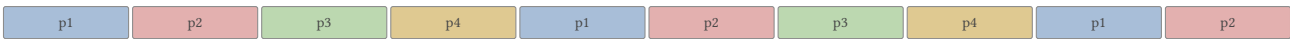
$[A, B] = \sum_{i < j} (a_i b_j - a_j b_i) [w_i, w_j]$

Every basis word w_i has a *degree* d_i (its length) and a *content* γ_i (its letter multiset). Brackets are *content-homogeneous*: $[w_i, w_j]$ decomposes only onto words of degree $d_i + d_j$ and content $\gamma_i + \gamma_j$. The kernel precomputes all *feasible* canonical pairs ($d_i + d_j \leq m$) into a flat table and groups them into *anagram units* — all pairs sharing $(t, \gamma) = (d_i + d_j, \gamma_i + \gamma_j)$:

- All of a unit’s decompositions land on *that unit’s own words* — the degree- t words with content γ . Two different units never write the same result word: *units are disjoint write regions*. No locks; no ordering across units (per-word accumulation order is preserved because a word is only ever written from inside its one unit).
- Entries within a unit are sorted by $p = d_i$ (so $q = t - p$ is forced) → contiguous p -runs degree-support gating keeps only the runs whose (p, q) degrees are backed by nonzeros in A and B .



schematic (4 of the 212 units): degree- t slice of the result vector — one cell per basis word, colour = owning anagram unit (a unit’s words are interleaved in memory with its neighbours’)



the unit’s pairs p1...p4, all sharing the same (t, γ) — they scatter only onto their own colour above

Real shape (measured, 2×12 grid): 747 basis words, **212 anagram units**, 1 694 feasible pairs; a unit owns ≈3.5 words and ≈8 pairs here. Units are packed into *bundles* of target size $\min(\max(\text{total_entries}/(2 \cdot \text{threads}), 16), 2048)$ — a unit is never split across bundles — and handed to rayon work-stealing, not a fixed assignment. Measured 2t assignment for 2×12: worker A took bundles of {444, 431} entries, worker B {499, 320} — balanced to within 7%.

But one kernel call ≠ just the sweep

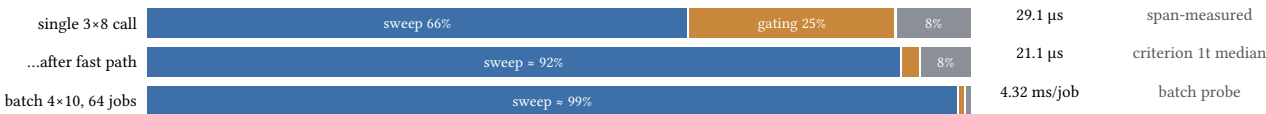


■ sweep 19.4 μs — the only parallel region (fork · work-stealing · join) ■ gating 7.4 μs (walk + presence + masks) ■ nonzero scan · alloc · bundle build · join ≈ 2.4 μs

one 3×8 kernel call, 29.1 μs at 1 thread (span-measured); segment widths to scale. The sweep lane shows both workers: each runs its own bundles over disjoint anagram units.

One call = *prologue* (① nonzero scan → presence bitsets + degree masks, ② gating walk over the whole unit table → active p-run segments) + *dispatch* (③ flatten segments into bundles, fork) + *sweep* (④ per entry: orientation presence tests, term $a_i b_j - a_j b_i$, scatter the stored decomposition) + ⑤ join. The prologue is serial per call but *memoized*: the GatingCache keys the active-segment list by the degree-mask pair, so the fold (cache reused across calls) pays ② once ever; and since today a *full-support fast path* (nonzero lists covering $0 \dots \text{degree_start}(\text{max_degree})$) skips the bitset scan and ② entirely — presence is all-ones and the active list is precomputed per table.

Proportion of the work: anagram-class commutations vs everything else



The commutations themselves dominate: 66% of a cold single call (25% was gating — fixed today), ≈92% after the fast path, ≈99% in the batch steady state the fold actually runs. The serial remainder is small *proportionally* but its *absolute* size decides small calls: all of 12×2 is 0.87 μs, so ≈2 μs of fork/join alone makes 2t slower (0.24×).

So why isn’t 2t = ½ the runtime, even at 99% parallel work?

$$T_2 \approx \underbrace{G + \delta}_{\text{serial: prologue + dispatch}} + \underbrace{\frac{P}{E_2}}_{\text{sweep on 2 threads}}$$

workload	serial s	E ₂ (sweep)	2t predicted	2t measured
single 3×8	8%	≈1.5×	1.42×	1.30×
batch 4×10	1%	≈1.8×	1.78×	1.74–1.80×

$E_2 < 2$ is a property of the sweep itself — measured with both workers 100% busy (perf at 9999 Hz; /proc tick deltas; 3.1 GHz on both cores; no idle cores; disjoint memory):

1. **Memory system, not dependencies**: the sweep is a streaming gather-scatter over the pair table and decomposition arrays; at 2t a batch burns ≈15% more cycles for identical work. Bigger bundles don’t help (cap 2048 → 2 M: no change), same-CCX pinning doesn’t help, page faults ruled out.
2. **Fork/join + bundle build** ≈ 1–2 μs per call — invisible at 4.3 ms/job (4×10 batch), fatal at 0.9 μs/job (12×2).
3. **History**: the bundle builder used to split units across bundles at every boundary — two workers +=-ing the same words (lost updates: flaky wrong results *and* worst-case cache-line sharing). Fixed today; the old oracle missed it because its grids fit inside a single bundle.

Take-away. The anagram-unit partition is sound and fully parallel. Where 2t lands is decided by (i) how much of the call is sweep — 66% → 92% → 99% for cold single call → fast path → batch — and (ii) the sweep’s memory-bound efficiency $E_2 \approx 1.5 \times$ (small grids) to $1.8 \times (4 \times 10)$. Measured outcomes: single calls 0.24–1.30× at 2t; batch 1.44–1.80× at 2t and up to 5.9× at 8t (commutator_kernel_batch bench, baseline postrace). Sources: criterion medians (today), tracing spans (pre-fastpath phase split), perf 9999 Hz profile, /proc per-thread accounting.